

Why is Singleton Class Sealed in C#?

In this article, I will discuss **Why Singleton Class Sealed in C#** with Examples. Please read our previous article discussing the [Singleton Design Pattern in C#](#) with Examples. At the end of this article, you will understand why we need to use the sealed keyword in the Singleton class, as we already have a private constructor, which will restrict the class for further inheritance.

Why Singleton Design Pattern?

As we already discussed, we need to use the Singleton Design Pattern in C# to ensure that only one instance of a particular class is available at any given point in time for the entire application.

Why is Singleton Class Sealed in C#?

In C#, it is common practice to declare Singleton classes as “sealed” to prevent them from being inherited and ensure that only one class instance can exist. Sealing a class means it cannot be inherited by other classes, making it impossible for anyone to create a subclass and potentially introduce multiple instances of the Singleton.

Let us understand Why the Singleton Class is Sealed in C# with an example. First, create the Singleton class without using the sealed keyword. Then, create another class named DerivedSingleton and Inherit the DerivedSingleton from the singleton class, as shown below.

```
using System;
namespace SingletonDemo
{
    public class Singleton
    {
        //This variable value will be increment by 1 each time the object
of the class is created
        private static int Counter = 0;
        //This variable is going to store the Singleton Instance
        private static Singleton Instance = null;
        //The following Static Method is going to return the Singleton
Instance
        public static Singleton GetInstance()
        {
            //If the variable instance is null, then create the Singleton
instance
            //else return the already created singleton instance
            //This version is not thread-safe
            if (Instance == null)
            {
                Instance = new Singleton();
            }
            //Return the Singleton Instance
            return Instance;
        }
        //Constructor is Private means, from outside the class we cannot
create an instance of this class
        private Singleton()
        {
            //Each Time the Constructor is called, increment the Counter
value by 1
            Counter++;
        }
    }
}
```

```

        Console.WriteLine("Counter Value " + Counter.ToString());
    }
    //The following can be accessed from outside of the class by using
the Singleton Instance
    public void PrintDetails(string message)
    {
        Console.WriteLine(message);
    }
}
public class DerivedSingleton : Singleton
{
}
}

```

At the moment, you will get the following compilation error. **Error CS0122 'Singleton.Singleton()' is inaccessible due to its protection level**

The above error is caused by our private constructor in the Singleton class. You might think that since we have a private constructor within the class, it is impossible to derive this class, so why do we need to apply the sealed keyword to it?

Let's move the DerivedSingleton class inside the Singleton class. When we move the DerivedSingleton class within the main Singleton class, it becomes a nested or child class of the main Singleton class, as shown below.

```

using System;
namespace SingletonDemo
{
    public class Singleton
    {
        //This variable value will be increment by 1 each time the object
of the class is created
        private static int Counter = 0;
        //This variable is going to store the Singleton Instance
        private static Singleton Instance = null;
        //The following Static Method is going to return the Singleton
Instance
        public static Singleton GetInstance()
        {
            //If the variable instance is null, then create the Singleton
instance
            //else return the already created singleton instance
            //This version is not thread-safe
            if (Instance == null)
            {
                Instance = new Singleton();
            }
            //Return the Singleton Instance
            return Instance;
        }
        //Constructor is Private means, from outside the class we cannot
create an instance of this class
        //But within the class, we can create an instance
        private Singleton()
        {

```

```

        //Each Time the Constructor is called, increment the Counter
value by 1
        Counter++;
        Console.WriteLine("Counter Value " + Counter.ToString());
    }
    //The following can be accessed from outside of the class by using
the Singleton Instance
    public void PrintDetails(string message)
    {
        Console.WriteLine(message);
    }
    //Creating Nested Derived Class inheriting from Singleton Class
    public class DerivedSingleton : Singleton
    {
    }
}
}

```

What is a Nested Class in C#?

Whenever we define a class within another class in C#, the inner class is called a nested or child class. Now, if we compile the program, we will not get any errors. Let us modify the main method of the program class to access the nested class, as shown below.

```

using System;
namespace SingletonDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Call the GetInstance static method to get the Singleton
Instance
            Singleton fromTeachaer = Singleton.GetInstance();
            fromTeachaer.PrintDetails("From Teacher");
            //Call the GetInstance static method to get the Singleton
Instance
            Singleton fromStudent = Singleton.GetInstance();
            fromStudent.PrintDetails("From Student");
            //Instantiating singleton from a Derived class.
            //This violates Singleton Pattern Pattern.

            Singleton.DerivedSingleton derivedObj = new
Singleton.DerivedSingleton();
            derivedObj.PrintDetails("From Derived");
            Console.ReadLine();
        }
    }
}

```

If you run the application, you will see the following output.

```
Counter Value 1
From Teacher
From Student
Counter Value 2
From Derived
```

The above output clearly shows that the counter value has incremented to 2, proving that the private constructor executed twice, creating multiple instances of the singleton class. So, by removing the sealed keyword, we can inherit the singleton class and create multiple objects of the singleton class. This violates singleton design principles. Let's make the Singleton class as sealed, as shown below, and then compile the program and see what happens.

```
using System;
namespace SingletonDemo
{
    public sealed class Singleton
    {
        //This variable value will be increment by 1 each time the object
        of the class is created
        private static int Counter = 0;
        //This variable is going to store the Singleton Instance
        private static Singleton Instance = null;
        //The following Static Method is going to return the Singleton
        Instance
        public static Singleton GetInstance()
        {
            //If the variable instance is null, then create the Singleton
            instance
            //else return the already created singleton instance
            //This version is not thread-safe
            if (Instance == null)
            {
                Instance = new Singleton();
            }
            //Return the Singleton Instance
            return Instance;
        }
        //Constructor is Private means, from outside the class we cannot
        create an instance of this class
        //But within the class, we can create an instance
        private Singleton()
        {
            //Each Time the Constructor is called, increment the Counter
            value by 1
            Counter++;
            Console.WriteLine("Counter Value " + Counter.ToString());
        }
        //The following can be accessed from outside of the class by using
        the Singleton Instance
        public void PrintDetails(string message)
        {
            Console.WriteLine(message);
        }
        //Creating Nested Derived Class inheriting from Singleton Class
    }
}
```

```

    public class DerivedSingleton : Singleton
    {
    }
}

```

We got an error when compiling the program, saying **we could not derive a sealed class**. So, from this point, we conclude that the private constructor in C# will help us prevent only external instantiations of the class, and the sealed keyword will prevent class inheritances.

The above Singleton implementation is not thread-safe. The way we implement the Singleton class allows two different threads simultaneously to evaluate the condition **if (instance == null)** and find it to be true. They both create instances that violate the singleton design pattern.

Why are Singleton Classes Sealed in C#?

The following are the reasons why Singleton classes are often sealed in C#:

- **Preventing Inheritance:** The Singleton pattern ensures that only one instance of a class exists throughout the application. If inheritance is allowed, a subclass can create additional instances, violating the Singleton Design Pattern. Making the class sealed ensures that no subclass can override methods or properties that could lead to additional instance creation.
- **Ensuring Singleton Integrity:** By declaring the class sealed, we ensure the integrity of the Singleton instance. If the class was not sealed, a derived class could add new static members, introduce a new access point to the Singleton instance, or even create new instances independently of the original Singleton control mechanism.
- **Simplifying Thread Safety:** Implementing thread safety in a Singleton can be challenging, especially in multi-threaded environments. Making the class sealed simplifies the synchronization logic, as we only need to consider the Singleton class itself and not any subclasses that might alter the behavior of the Singleton instance.
- **Optimization:** The compiler can make certain optimizations, knowing that the class cannot be inherited. For example, method calls to members of a sealed class can sometimes be faster because the method dispatching process is simpler and can be more efficiently optimized during JIT (Just-In-Time) compilation.

In the next article, I will discuss [how to Handle Thread Safety in the Singleton Class](#), as the current version of singleton implementation can create multiple instances of the singleton class in multi-threaded environments. I explain **Why the Singleton Class Sealed in C#** in this article with Examples. I hope this article will help you to understand Why the Singleton Class is Sealed in C# with Examples.